instituto superior de engenharia de lisboa

LMATE

Programação

Inverno 2026-2027

Trabalho 14c

1

Ciclo `for`

Um ciclo `for` usa:

- uma variável
- e uma sequência de valores

e tem:

- um bloco de código

Para cada valor na sequência de valores, por ordem:

1. a variável é inicializada com o valor em causa
2. o bloco de código é executado

O bloco de código tem que estar indentado com quatro espaço relativamente à palavra `for`.
Ao bloco de código também se chama o corpo do ciclo `for`.
No final do ciclo `for` a variável usada fica com o último valor da sequência.

Exemplo (no ficheiro `ciclo_for_exemplo.py`)

```
contador = 1
for preço in [97, 70, 15, 38, 92, 30, 86, 68, 6, 80]:
    print('a executar o corpo do ciclo for')
    print('número de vezes = ', contador )
    print('preço =', preço)
    contador = contador + 1
print('fim') # esta linha está fora do ciclo for. experimente indentar
             # esta linha. qual é a diferença?
print(preço) # a variável do ciclo fica definida e com o último valor
             # da sequência
```

Output

```
a executar o corpo do ciclo for
número de vezes =  1
preço = 97
a executar o corpo do ciclo for
número de vezes =  2
preço = 70
a executar o corpo do ciclo for
número de vezes =  3
preço = 15
a executar o corpo do ciclo for
número de vezes =  4
preço = 38
a executar o corpo do ciclo for
número de vezes =  5
preço = 92
a executar o corpo do ciclo for
número de vezes =  6
preço = 30
a executar o corpo do ciclo for
número de vezes =  7
preço = 86
a executar o corpo do ciclo for
número de vezes =  8
preço = 68
a executar o corpo do ciclo for
número de vezes =  9
preço = 6
a executar o corpo do ciclo for
número de vezes =  10
preço = 80
fim
80
```

Função `range`

- A indexação de listas é tão útil e tão frequente que existe uma função para gerar sequências de índices. É a função *built-in* `range`. *Range* em português quer dizer gama, gama de valores.
- Quando usada em ciclos `for`, para indexar listas, é frequente ser usada em conjunto com a função `len`, para tornar o ciclo `for` independente do tamanho da lista.

Exemplo (no ficheiro `range_exemplo.py`)

```
lista_de_preços    = [97, 70, 15, 38, 92, 30, 86, 68, 6,  80]
lista_de_descontos = [10, 15, 5,  25, 10, 10, 35, 25, 10, 5]

for índice in range(len(lista_de_preços)):

    print('índice =', índice)
    preço = lista_de_preços[índice]
    desconto = lista_de_descontos[índice]
    print('preço =', preço, 'desconto =', desconto)
```

Output

```
índice = 0
preço = 97 desconto = 10
índice = 1
preço = 70 desconto = 15
índice = 2
preço = 15 desconto = 5
índice = 3
preço = 38 desconto = 25
índice = 4
preço = 92 desconto = 10
índice = 5
preço = 30 desconto = 10
índice = 6
preço = 86 desconto = 35
índice = 7
preço = 68 desconto = 25
índice = 8
preço = 6 desconto = 10
índice = 9
preço = 80 desconto = 5
```

- A função `range` pode ser usada com um, dois ou três argumentos. Todos os argumentos têm que ser números inteiros.

Um argumento (é a utilização mais frequente)

- Quando se usa um argumento `n`, a função `range` cria a sequência de zero até `n-1`. É a gama de índices apropriada para indexar uma lista com `n` elementos. O valor `n` não é incluido na sequência. Porque não é um índice válido de uma lista com `n` elementos.

Exemplo (no ficheiro `range_exemplo_um_argumento.py`)

```
# gera a sequência de zero a quatro
for índice in range(5):
    print(índice)
```

Output

```
0
1
2
3
4
```

Dois argumentos

- Com dois argumentos, `n1` e `n2`, especifica-se o ínico e o fim da sequência a criar, `n1` e `n2`, respetivamente. A função `range` cria a sequência de `n1` até `n2-1`.

Exemplo (no ficheiro `range_exemplo_dois_argumentos.py`)

```
# gera a sequência de cinco a nove
for índice in range(5, 10):
    print(índice)
```

Output

```
5
6
7
8
9
```

Três argumentos

- Com três argumentos, `n1`, `n2` e `n3`, especifica-se o ínico e o fim da sequência a criar, `n1` e `n2`, respetivamente, e também o incremento, ou o passo, entre valores consecutivos.

Exemplo (no ficheiro `range_exemplo_tres_argumentos.py`)

```
# gera a sequência de cinco a treze, de dois em dois
for índice in range(5, 15, 2):
    print(índice)
```

Output

```
5
7
9
11
13
```

Indique se as afirmações que se seguem são verdadeiras ou falsas.

1.1

Considere o programa que se segue. Copie o programa exatamente como está para o ficheiro `programa_5.py`.

```
c = [12, 7, 25, 40]
for f in range(len(c)):
    print(f)
```

O *output* da execução deste programa é:

```
0
1
2
3
```

1.2

Considere o programa que se segue. Copie o programa exatamente como está para o ficheiro `programa_6.py`.

```
v = [12, 7, 25, 40]
for x in range(len(v)):
    print(v[x])
```

O *output* da execução deste programa é:

```
12
7
25
40
```

1.3

Considere o programa que se segue. Copie o programa exatamente como está para o ficheiro `programa_4.py`.

```
t = [12, 7, 25, 40]
for t in d:
    print(d)
```

A execução deste programa dá erro.

1.4

Considere o programa que se segue. Copie o programa exatamente como está para o ficheiro `programa_2.py`.

```
for i in [12, 7, 25, 40]:
print(i)
```

A execução deste programa dá erro.

1.5

Considere o programa que se segue. Copie o programa exatamente como está para o ficheiro `programa_3.py`.

```
a = [12, 7, 25, 40]
for a in a:
    print(a)
print(len(a))
```

A execução deste programa dá erro.

1.6

Considere o programa que se segue. Copie o programa exatamente como está para o ficheiro `programa_1.py`.

```
for e in [12, 7, 25, 40]
    print(e)
```

A execução deste programa dá erro.

2

Listas

- Para além dos valores elementares como as *strings*, os números inteiros, os números decimais, ou os valores booleanos, existem em Python outros valores. É o caso das listas.
- Uma lista é um valor que permite armazenar uma sequência ordenada de outros valores.
- Em Python todos os valores são de algum tipo. As listas são do tipo `list` (que em português quer dizer "lista").

Criar listas

- Uma lista armazena uma sequência ordenada de valores. Aos valores armazenados nas listas chamam-se elementos.
- Para criar uma lista colocam-se os seus elementos entre parentsis retos, separados por vírgulas.

Exemplo (no ficheiro `listas_exemplo_criar.py`)

Escreva e execute este programa. Confirme o seu *output*.

```
lista_de_precos = [97, 70, 15, 38, 92, 30, 86, 68, 6, 80]
print(lista_de_precos)
```

Output

```
[97, 70, 15, 38, 92, 30, 86, 68, 6, 80]
```

Elementos de vários tipos

- Os elementos não têm que ser todos do mesmo tipo.

Exemplo (no ficheiro `listas_exemplo_criar_varios_tipos_elementos.py`)

Escreva e execute este programa. Confirme o seu *output*.

```
# produto
# nome:     Farinha (tipo string)
# preço:    1.23    (tipo float)
# desconto: 10      (tipo int)
# em stock: True    (tipo bool)

# esta lista representa o produto "Farinha"
produto = ['Farinha', 1.23, 10, True]
print(produto)
```

Output

```
['Farinha', 1.23, 10, True]
```

Lista vazia

- Pode criar-se uma lista vazia, isto é, sem qualquer elemento.

Exemplo (no ficheiro `listas_exemplo_lista_vazia.py`)

Escreva e execute este programa. Confirme o seu *output*.

```
# uma lista para armazenar os produtos esgotados
# como não há nenhum produto esgotado a lista está vazia
produtos_esgotados = [] # a lista vazia
print(produtos_esgotados)
```

Output

```
[]
```

Indexar listas

- Os valores armazenados nas listas podem ser obtidos a partir dos seus índices na sequência. Chama-se a isto indexar a lista.
- Os índices das listas são números inteiros e iniciam-se em zero.
- Isto quer dizer que o índice do primeiro elemento na lista é zero, o do segundo é um, o do terceiro é dois, e assim por diante.

Exemplos (no ficheiro `listas_exemplos_indexar.py`)

Escreva e execute este programa. Confirme o seu *output*.

```
lista_de_precos   = [97, 70, 15, 38, 92, 30, 86, 68, 6, 80]
primeiro_elemento = lista_de_precos[0] # o primeiro elemento está no índice 0
segundo_elemento  = lista_de_precos[1] # o segundo elemento está no índice 1
ultimo_elemento   = lista_de_precos[9] # como esta lista tem 10 elementos o
                                       # último elemento está no índice 9
                    # (porque os índices começam em zero)

print(lista_de_precos)
print(primeiro_elemento)
print(segundo_elemento)
print(ultimo_elemento)
```

Output

```
[97, 70, 15, 38, 92, 30, 86, 68, 6, 80]
97
70
80
```

Erro - indexar elemento não existente

- Se se tentar indexar uma lista, com um índice de um elemento que não existe, o interpretador Python dá erro.
- (Existe uma gama de índices dos elementos da lista. Quando se tenta usar um índice de um elemento que não existe, esse índice está fora da gama dos índices dos elementos da lista. E por isso é que o erro que o interpretador Python dá é *"IndexError: list index out of range"*.)

Exemplo (no ficheiro `listas_exemplos_erro.py`)

Escreva e execute este programa. Confirme o seu *output*.

```
lista_de_precos  = [97, 70, 15, 38, 92, 30, 86, 68, 6, 80]
print(lista_de_precos)
elemento = lista_de_precos[8]
print(elemento)
elemento = lista_de_precos[9]
print(elemento)
# a lista só vai até ao índice 9. tem que ser corrigida. porque não existe nenhum
# elemento no índice 10 de uma lista com 10 elementos. porque os
# índices começam em zero. o índice do último elemento é 9
elemento = lista_de_precos[10]
print(elemento)
```

Output

```
[97, 70, 15, 38, 92, 30, 86, 68, 6, 80]
6
80
Traceback (most recent call last):
  File "/home/jm/develop/202309_hello_python_world_v2/pt/listas/listas_exemplos_erro.py", line 10, in <module>
    elemento = lista_de_precos[10]
               ~~~~~~~~~~~~~~~^^^^
IndexError: list index out of range
```

Indexar a partir do fim

- Podem usar-se índices negativos.
- Quando se usam índices negativos a indexação é efetuada a partir do fim da lista.
- O último elemento da lista está no índice `-1`. O penúltimo está no índice `-2`. E assim sucessivamente (andando para trás).

Muito útil

- Como o índice do último elemento das listas é `-1`, isso permite obter o último elemento das listas sem ter que se saber o seu tamanho. E isso é muito útil. Independentemente do tamanho das listas o último elemento está sempre no índice `-1`. (Assim como independentemente do tamanho das listas o primeiro elemento está sempre no índice `0`.)

Exemplos (no ficheiro `listas_exemplos_indices_negativos.py`)

Escreva e execute este programa. Confirme o seu *output*.

```
lista_de_precos   = [97, 70, 15, 38, 92, 30, 86, 68, 6, 80]
# o último elemento está no índice -1
ultimo_elemento   = lista_de_precos[-1]
# o penúltimo elemento está no índice -2
penultimo_elemento = lista_de_precos[-2]
# o primeiro elemento está no índice -10 quando a partir do fim, o
# primeiro elemento está no índice -10
primeiro_elemento = lista_de_precos[-10]
print(lista_de_precos)
print(último_elemento)
print(penúltimo_elemento)
print(primeiro_elemento)
# a lista só vai até ao índice -10. tem que ser corrigida. numa lista com
# 10 elementos não existe o elemento no índice -11
elemento = lista_de_precos[-11]
print(elemento)
```

Output

```
[97, 70, 15, 38, 92, 30, 86, 68, 6, 80]
80
97
Traceback (most recent call last):
  File "/home/jm/develop/202309_hello_python_world_v2/pt/listas/listas_exemplos_indices_negativos.py", line 16, in <module>
    elemento = lista_de_precos[-11]
               ~~~~~~~~~~~~~~~^^^^
IndexError: list index out of range
```

Alterar listas

- Os elementos armazenados nas listas podem ser alterados.
- As listas são alteráveis.
- O termo usado em contexto Python é que as listas são mutáveis.

Alterar usando o índice do elemento

- Pode-se alterar cada um dos elementos de uma lista através do índice respetivo.

Exemplos (no ficheiro `listas_exemplos_alterar_usando_indice.py`)

```
produto = ['Farinha', 1.23, 10, True]
print(produto)
produto[0] = 'Arroz'
print(produto)
produto[1] = 0.80
print(produto)
produto[2] = 5
print(produto)
produto[3] = False
print(produto)
```

Output

```
['Farinha', 1.23, 10, True]
['Arroz', 1.23, 10, True]
['Arroz', 0.8, 10, True]
['Arroz', 0.8, 5, True]
['Arroz', 0.8, 5, False]
```

Alterar usando funções

- Podem acrescentar-se elementos ao fim das listas usando a função `append`. Podem inserir-se elementos num dado índice usando a função `insert`. Pode remover-se um elemento através do seu índice usando a função `pop`. A função `pop` retorna o elemento removido.
- As funções `append`, `insert` e `pop` só existem no contexto das listas. Para serem chamadas usa-se o operador ponto `.` a seguir à variável que armazena uma lista, seguido da chamada à função pretendida. A função é executada sobre a lista armazenada na variável usada. Trata-se de um mecanismo relativo à programação orientada por objetos que veremos mais à frente.

Exemplos (no ficheiro `listas_exemplos_alterar_usando_funcoes.py`)

```
lista_de_preços = [97, 70, 15, 38, 92, 30, 86, 68, 6, 80]
print(lista_de_preços)
# acrescentar um preço no final da lista
lista_de_preços.append(100)
print(lista_de_preços)
# inserir um preço no terceiro elemento
lista_de_preços.insert(2, 200)
print(lista_de_preços)
# remover o primeiro preço
preço_removido = lista_de_preços.pop(0)
print(lista_de_preços)
print(preço_removido)
```

Output

```
[97, 70, 15, 38, 92, 30, 86, 68, 6, 80]
[97, 70, 15, 38, 92, 30, 86, 68, 6, 80, 100]
[97, 70, 200, 15, 38, 92, 30, 86, 68, 6, 80, 100]
[70, 200, 15, 38, 92, 30, 86, 68, 6, 80, 100]
97
```

Número de elementos de listas

- Pode obter-se o número de elementos de uma lista usando a função *built-in* `len`. São as três primeiras letras da palavra inglesa *length*, que quer dizer comprimento. Neste contexto, a função `len` retorna o número de elementos da lista, ou o comprimento da lista.

Exemplos (no ficheiro `listas_exemplos_len.py`)

```
lista_de_preços = [97, 70, 15, 38, 92, 30, 86, 68, 6, 80]
print(lista_de_preços)
# número de elementos da lista
lista_de_preços_comprimento = len(lista_de_preços)
print(lista_de_preços_comprimento)
produto = ['Farinha', 1.23, 10, True]
print(produto)
# número de elementos da lista
produto_comprimento = len(produto)
print(produto_comprimento)
```

Output

```
[97, 70, 15, 38, 92, 30, 86, 68, 6, 80]
10
['Farinha', 1.23, 10, True]
4
```

Indique se as afirmações que se seguem são verdadeiras ou falsas.

2.1

Considere o programa que se segue. Copie o programa exatamente como está para o ficheiro `programa_3.py`.

```
n = [12, 7, 25, 40]
print(n[4])
```

A execução deste programa dá erro.

2.2

Considere o programa que se segue. Copie o programa exatamente como está para o ficheiro `programa_6.py`.

```
i = [12, 7, 25, 40]
print(i)
i.append(30)
print(i)
c = []
print(c)
c.append(30)
print(c)
```

A execução deste programa dá erro.

2.3

Considere o programa que se segue. Copie o programa exatamente como está para o ficheiro `programa_7.py`.

```
z = [12, 7, 25, 40]
print(len(z))
print(z)
print(z[len(z)])
```

A execução deste programa não dá erro.

2.4

Considere o programa que se segue. Copie o programa exatamente como está para o ficheiro `programa_5.py`.

```
f = [12, 7, 25, 40]
print(f)
g = f[1]
print(g)
f[1] = 30
print(f)
```

O *output* da execução deste programa é:

```
[12, 7, 25, 40]
7
[12, 30, 25, 40]
```

2.5

Considere o programa que se segue. Copie o programa exatamente como está para o ficheiro `programa_4.py`.

```
j = [12, 7, 25, 40, 18]
print(j[-1])
print(j[-2])
print(j[-4])
```

O *output* da execução deste programa é:

```
18
40
7
```

2.6

Considere o programa que se segue. Copie o programa exatamente como está para o ficheiro `programa_2.py`.

```
x = []
print(x)
```

A execução deste programa não dá erro.

2.7

Considere o programa que se segue. Copie o programa exatamente como está para o ficheiro `programa_1.py`.

```
m = ['Arroz', 0.85, 3, True]
print(m)
```

A execução deste programa não dá erro.

3

Exercício 8 - Tabela de compras

- Considere a tabela que se segue:

Compras	Produto 1	Produto 2	...	Produto m
Cliente 1	0	2		0
Cliente 2	3	0		0
...				
Cliente n	0	0		0

- Há n clientes. A tabela tem n linhas. Uma para cada cliente.
- Há s produtos. A tabela tem s colunas. Uma para cada produto.
- A tabela permite registar que quantidades de produtos foram compradas por cada cliente.

- Por exemplo, o cliente 2 comprou 3 unidades do produto 1. O cliente 1 comprou 2 unidades do produto 2. Etc.
- Escreva:
  1. Uma função que receba como argumentos um número de linhas n, um número de colunas s, um valor inicial v e retorne uma tabela criada. Todos os números na tabela deverão ser inicializados a zero.
  2. Uma função que receba como argumento uma tabela, e que faça *output* da tabela para o ecrã. Esta função não retorna nada.
  3. Uma função que receba como argumentos, uma tabela, um número de linha e um número de coluna. E retorne o valor que está na tabela na linha e coluna cujos números foram recebidas como argumentos. Note-se que os números de linha e de colunas se iniciam em um.
  4. Uma função que receba como argumentos, uma tabela, um número de linha, um número de coluna, e um valor. Esta função coloca o valor recebido como argumento na tabela recebida como argumento. Na linha e coluna cujos números também recebeu como argumentos. Note-se mais uma vez que os números de linha e de colunas se iniciam em um. Esta função não retorna nada.
  5. Alguns exemplos que ilustrem o funcionamento destas funções.

Solução

- Usar listas com listas.
- A tabela é uma lista com linhas. Cada linha é, por sua vez, uma lista com números. Todas as linhas têm o mesmo número de elementos.

Primeira versão

- Para um número fixo de 3 linhas, e para um número fixo de 5 colunas.

Programa (no ficheiro `exercicio_8_tabela_de_compras.py`**)**

Escreva e execute este programa. Confirme o seu *output*.

```
# função para criar uma tabela
def cria_tabela():

    tabela = [
        [0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0]
    ]

    return tabela

# função para fazer output de uma tabela
def mostra_tabela(tabela):

    for indice_linha in range(3):
        linha = tabela[indice_linha]
        print(linha)

# função para retornar o número numa linha e coluna de uma tabela
def retorna_valor(tabela, número_linha, número_coluna):

    indice_linha = número_linha - 1
    indice_coluna = número_coluna - 1

    valor = tabela[indice_linha][indice_coluna]

    return valor

# função para guardar numa linha e coluna um valor
def altera_valor(tabela, número_linha, número_coluna, valor):

    indice_linha = número_linha - 1
    indice_coluna = número_coluna - 1

    tabela[indice_linha][indice_coluna] = valor

# exemplo
tabela_1 = cria_tabela()
print('tabela1')
mostra_tabela(tabela_1)
print('valores iniciais')
valor_1_1 = retorna_valor(tabela_1, 2, 1)
print('linha 2, coluna 1, valor = ', valor_1_1)
valor_1_2 = retorna_valor(tabela_1, 1, 2)
print('linha', 1, 'coluna', 2, 'valor', valor_1_2)
altera_valor(tabela_1, 2, 1, 3)
altera_valor(tabela_1, 1, 2, 2)
print('a alterar valores... fim.')
print('valores alterados')
valor_2_1 = retorna_valor(tabela_1, 2, 1)
print('linha 2, coluna 1, valor = ', valor_2_1)
valor_1_2 = retorna_valor(tabela_1, 1, 2)
print('linha', 1, 'coluna', 2, 'valor', valor_1_2)
print('tabela1')
mostra_tabela(tabela_1)
```

Output

```
tabela1
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
valores iniciais
linha 2 coluna 1 valor 0
linha 1 coluna 2 valor 0
a alterar valores... fim.
valores alterados
linha 2 coluna 1 valor 3
linha 1 coluna 2 valor 2
tabela
[0, 2, 0, 0, 0]
[3, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
```

Segunda versão

- Para um número de linhas e para um número de colunas quaisquer.

Programa (no ficheiro `exercicio_8_tabela_de_compras_v2.py`**)**

Escreva e execute este programa. Confirme o seu *output*.

```
# função para criar uma tabela
def cria_tabela(número_de_linhas, número_de_colunas):

    tabela = []

    for indice_linha in range(número_de_linhas):
        nova_linha = []
        for indice_coluna in range(número_de_colunas):
            nova_linha.append(0)
        tabela.append(nova_linha)

    return tabela

# função para fazer output de uma tabela
def mostra_tabela(tabela):

    for indice_linha in range(len(tabela)):
        linha = tabela[indice_linha]
        print(linha)

# função para retornar o número numa linha e coluna de uma tabela
def retorna_valor(tabela, número_linha, número_coluna):

    indice_linha = número_linha - 1
    indice_coluna = número_coluna - 1

    valor = tabela[indice_linha][indice_coluna]

    return valor

# função para guardar numa linha e coluna um valor
def altera_valor(tabela, número_linha, número_coluna, valor):

    indice_linha = número_linha - 1
    indice_coluna = número_coluna - 1

    tabela[indice_linha][indice_coluna] = valor

# exemplo
tabela = cria_tabela(7, 10)
print('tabela7x10')
mostra_tabela(tabela)
print('valores iniciais')
valor_2_1 = retorna_valor(tabela, 2, 1)
print('linha 2, coluna 1, valor = ', valor_2_1)
valor_1_2 = retorna_valor(tabela, 1, 2)
print('linha', 1, 'coluna', 2, 'valor', valor_1_2)
print('a alterar valores... fim.')
altera_valor(tabela, 2, 1, 3)
altera_valor(tabela, 1, 2, 2)
print('a alterar valores... fim.')
print('valores alterados')
valor_2_1 = retorna_valor(tabela, 2, 1)
print('linha 2, coluna 1, valor = ', valor_2_1)
valor_1_2 = retorna_valor(tabela, 1, 2)
print('linha', 1, 'coluna', 2, 'valor', valor_1_2)
print('tabela')
mostra_tabela(tabela)
```

Output

```
tabela
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
valores iniciais
linha 2 coluna 1 valor 0
linha 1 coluna 2 valor 0
a alterar valores... fim.
a alterar valores... fim.
valores alterados
linha 2 coluna 1 valor 3
linha 1 coluna 2 valor 2
tabela
[0, 2, 0, 0, 0, 0, 0, 0, 0, 0]
[3, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Listas com listas

Os elementos das listas podem ser de qualquer tipo, incluindo do tipo listas. Isto quer dizer que as listas podem armazenar outras listas.

Exemplos (no ficheiro `listas_exemplos_com_listas.py`**)**

```
# uma lista com listas
lista = [
    [0, 2, 0, 0, 0],
    [3, 0, 0, 0, 0],
    [1, 0, 0, 0, 1]
]

print(lista)
```

Output

```
[[0, 2, 0, 0, 0], [3, 0, 0, 0, 0], [1, 0, 0, 0, 1]]
```

Indexação de listas com listas

A indexação de listas com listas pode ser efetuada em cadeia, isto é, se o resultado da indexação de uma lista A for uma lista, então essa indexação pode ser seguida de nova indexação.

Exemplos (no ficheiro `listas_exemplos_com_listas_indexacao.py`**)**

```
# uma lista com listas
lista = [
    [0, 2, 0, 0, 0],
    [3, 0, 0, 0, 0],
    [1, 0, 0, 0, 1]
]

print(lista)
print(lista[0])
print(lista[0][0])
print(lista[0][1])
print(lista[0][2])
print(lista[1])
print(lista[1][0])
print(lista[2])
print(lista[2][0])
print(lista[2][4])
print(lista[3])
print(lista[3][0])
print(lista[-1][-1])
```

Output

```
[[0, 2, 0, 0, 0], [3, 0, 0, 0, 0], [1, 0, 0, 0, 1]]
[0, 2, 0, 0, 0]
0
2
0
[3, 0, 0, 0, 0]
3
[1, 0, 0, 0, 1]
1
1
```

Indexação de listas com listas com ciclos `for`

Todas as listas numa estrutura de listas com listas podem ser indexadas com ciclos `for`. Porque são listas.

Exemplos (no ficheiro `listas_exemplos_com_listas_ciclo_for.py`**)**

```
# uma lista com listas
lista = [
    [0, 2, 0, 0, 0],
    [0, 0, 7, 0, 0],
    [1, 0, 0, 0, 1]
]

print(lista)

for linha in lista:
    print('linha = ', linha)
    for valor in linha:
        print('valor = ', valor)

# o mesmo mas agora com índices
print('##########################')
print('agora com índices #####')
print('##########################')

for indice_linha in range(len(lista)):
    linha = lista[indice_linha]
    print('linha = ', linha)
    for indice_coluna in range(len(linha)):
        valor = linha[indice_coluna]
        print('valor = ', valor)
```

Output

```
[[0, 2, 0, 0, 0], [0, 0, 7, 0, 0], [1, 0, 0, 0, 1]]
linha =  [0, 2, 0, 0, 0]
valor =  0
valor =  2
valor =  0
valor =  0
valor =  0
linha =  [0, 0, 7, 0, 0]
valor =  0
valor =  0
valor =  7
valor =  0
valor =  0
linha =  [1, 0, 0, 0, 1]
valor =  1
valor =  0
valor =  0
valor =  0
valor =  1
##########################
##### agora com índices #####
##########################
linha =  [0, 2, 0, 0, 0]
valor =  0
valor =  2
valor =  0
valor =  0
valor =  0
linha =  [0, 0, 7, 0, 0]
valor =  0
valor =  0
valor =  7
valor =  0
valor =  0
linha =  [1, 0, 0, 0, 1]
valor =  1
valor =  0
valor =  0
valor =  0
valor =  1
```

Para fazer

Considere o programa Python 3 que se segue, assim como o seu *output*. Copie o programa para um ficheiro com você. Os `import`s, bem como as instruções `seed`, `randint` e `choices`, e a função `criar_matriz_aleatoria`, destinam-se exclusivamente à geração de valores pseudoaleatórios, de forma repetível.

```
from random import seed
from random import randint
from random import choices

seed(5000)

def print_matriz(matriz):
    número_linhas = len(matriz)
    for indice_linha in range(número_linhas):
        print(número_linhas)
        print(indice_linha)
        linha = matriz[indice_linha]
        print(linha)
        print('----------')

exemplo = [
    [45, 35, 90, 80, 50],
    [80, 70, 90, 80, 10],
    [10, 10, 10, 20, 10]
]

print_matriz(exemplo)
print(exemplo[0][0])
print(exemplo[0][1])
print(exemplo[0][2])
print(exemplo[0][3])
print(exemplo[0][4])

def criar_matriz_aleatoria(l, c):
    # l = número de linhas, c = número de colunas
    número_colunas = randint(10, 15)
    matriz = []
    for n in range(número_linhas):
        matriz.append(choices(range(10, 100), k=número_colunas))
    return matriz

a_1 = criar_matriz_aleatoria(7)
a_2 = criar_matriz_aleatoria(7)
a_3 = criar_matriz_aleatoria(7)
a_4 = criar_matriz_aleatoria(7)
a_5 = criar_matriz_aleatoria(7)

print('a1 para verificação da geração de valores pseudoaleatórios')
print(a_1[0][0])
print(a_1[0][1])
print(a_1[0][2])
print(a_1[0][3])
print(a_1[0][4])
```

Nesta programa uma matriz é representada por uma lista de linhas. Cada linha é, por sua vez, uma lista de números. Todas as linhas têm o mesmo número de elementos. Trata-se portanto de uma matriz bidimensional de números.

A função `print_matriz` mostra o número de linhas da matriz, o número de colunas da matriz e, depois, cada uma das linhas da matriz.

O *output* deste programa é o que se segue.

```
3
0
[45, 35, 90, 80, 50]
[80, 70, 90, 80, 10]
[10, 10, 10, 20, 10]
----------
[80, 70, 90, 80, 10]
1
0
45
35
90
80
50
a1 para verificação da geração de valores pseudoaleatórios
79
69
49
49
```

Indique se as afirmações que se seguem são verdadeiras ou falsas.

3.1

Acrescentando ao fim do programa as instruções que se seguem:

```
print_matriz(s_1)
print(s_1[3])
```

o *output* produzido por estas instruções é:

```
12
14
[29, 51, 30, 44, 72, 95, 65, 40, 47, 20, 30, 95, 80, 90]
[63, 22, 10, 78, 13, 10, 59, 88, 50, 16, 47, 58, 46, 76]
[32, 60, 36, 30, 95, 49, 33, 83, 31, 40, 92, 29, 53, 53]
[49, 30, 30, 30, 12, 92, 71, 92, 79, 21, 25, 86, 44, 17]
[43, 19, 25, 71, 34, 68, 56, 97, 54, 38, 55, 71, 49, 49]
[87, 66, 72, 52, 92, 78, 28, 61, 56, 45, 87, 46, 27, 25]
[45, 64, 20, 97, 88, 88, 19, 30, 80, 18, 21, 25, 88, 26]
[43, 68, 34, 85, 12, 14, 67, 99, 57, 46, 10, 45, 97, 34]
[21, 58, 26, 10, 63, 77, 57, 59, 44, 74, 37, 54, 53, 59]
[40, 22, 49, 69, 69, 77, 12, 59, 80, 42, 27, 25, 17, 63]
[85, 59, 58, 46, 45, 73, 67, 98, 95, 16, 92, 92, 61, 20]
[74, 38, 58, 38, 47, 50, 15, 37, 19, 47, 98, 98, 92, 43]
----------
[32, 60, 36, 30, 95, 49, 33, 83, 31, 40, 92, 29, 53, 53]
```

3.2

Acrescentando ao fim do programa as instruções que se seguem:

```
print_matriz(s_4)
print(s_4[2][13])
```

o *output* produzido por estas instruções é:

```
14
14
[82, 12, 26, 72, 70, 76, 65, 38, 47, 60, 75, 31, 46, 17]
[36, 34, 23, 37, 33, 74, 61, 45, 92, 42, 15, 13, 80, 48]
[67, 14, 33, 63, 94, 65, 54, 86, 18, 82, 32, 23, 96, 55]
[80, 67, 63, 18, 27, 17, 89, 93, 85, 42, 52, 36, 12, 29]
[98, 52, 28, 12, 33, 77, 88, 16, 34, 56, 50, 33, 62, 63]
[95, 66, 99, 67, 35, 88, 41, 45, 49, 67, 71, 69, 39, 55]
[31, 53, 58, 35, 77, 47, 10, 43, 39, 65, 59, 20, 55, 89]
[14, 81, 77, 56, 46, 87, 18, 66, 70, 22, 30, 86, 97, 68]
[99, 16, 59, 50, 37, 17, 34, 67, 14, 90, 84, 52, 42, 77]
[17, 74, 33, 49, 92, 71, 14, 89, 75, 61, 23, 17, 91, 26]
[88, 81, 73, 86, 92, 91, 51, 95, 69, 90, 14, 91, 97, 79]
[98, 19, 98, 69, 91, 69, 42, 83, 26, 72, 35, 95, 49, 97]
[92, 13, 30, 19, 92, 30, 87, 10, 35, 32, 53, 69, 95, 82]
[42, 22, 35, 75, 69, 79, 32, 81, 45, 85, 61, 92, 43, 35]
----------
55
```

3.3

Acrescentando ao fim do programa as instruções que se seguem:

```
print_matriz(s_3)
print(s_3[1][0])
```

o *output* produzido por estas instruções é:

```
14
10
[69, 85, 60, 90, 81, 46, 26, 19, 18, 93]
[93, 59, 23, 30, 67, 87, 47, 78, 60, 53]
[99, 43, 56, 61, 66, 31, 57, 42, 78, 85]
[51, 52, 93, 81, 64, 61, 79, 25, 40, 53]
[60, 98, 32, 68, 55, 99, 34, 79, 88, 66]
[24, 63, 87, 95, 61, 38, 46, 69, 39, 89]
[32, 55, 40, 56, 55, 33, 59, 67, 72, 33]
[41, 44, 19, 21, 36, 25, 83, 60, 89, 79]
[59, 90, 95, 92, 77, 19, 69, 69, 69, 50]
[47, 15, 68, 13, 33, 94, 85, 69, 81, 70]
[96, 76, 54, 12, 40, 36, 73, 26, 55, 33]
[71, 87, 19, 98, 95, 56, 18, 23, 51, 57]
[46, 53, 54, 45, 35, 89, 74, 86, 74, 12]
[64, 42, 72, 28, 93, 12, 97, 43, 74, 26]
----------
93
```

3.4

Acrescentando ao fim do programa as instruções que se seguem:

```
print_matriz(s_5)
print(s_5[13][4])
```

o *output* produzido por estas instruções é:

```
14
12
[67, 85, 62, 99, 10, 20, 58, 22, 43, 52, 32, 61]
[27, 95, 45, 13, 28, 89, 44, 28, 11, 69, 99, 19]
[40, 27, 53, 53, 26, 72, 12, 18, 77, 64, 30, 13]
[99, 35, 49, 17, 74, 40, 33, 48, 75, 98, 15, 36]
[74, 80, 62, 31, 63, 64, 25, 82, 51, 65, 91, 13]
[81, 29, 38, 83, 89, 59, 98, 22, 33, 85, 78, 51]
[11, 74, 11, 59, 60, 75, 53, 54, 24, 31, 12, 86]
[89, 54, 87, 81, 59, 72, 86, 43, 77, 60, 59, 46]
[75, 81, 93, 44, 46, 97, 27, 47, 85, 67, 69, 50]
[15, 91, 70, 75, 68, 38, 98, 50, 88, 95, 25, 81]
[32, 49, 43, 66, 45, 68, 91, 42, 23, 19, 64, 60]
[41, 75, 61, 26, 40, 81, 55, 65, 48, 43, 88, 64]
[23, 70, 93, 26, 61, 13, 33, 97, 73, 33, 51, 84]
[18, 36, 47, 24, 43, 63, 57, 45, 56, 83, 88, 87]
----------
43
```

3.5

Acrescentando ao fim do programa as instruções que se seguem:

```
print_matriz(s_2)
for linha in s_2:
    print(linha[8])
```

o *output* produzido por estas instruções é:

```
10
14
[79, 38, 50, 76, 19, 39, 72, 99, 25, 36, 45, 26, 54, 38]
[83, 81, 77, 43, 51, 39, 17, 74, 15, 57, 91, 20, 29, 73]
[17, 60, 17, 25, 78, 67, 73, 32, 99, 58, 59, 78, 44, 54]
[79, 16, 19, 12, 85, 71, 60, 92, 29, 25, 91, 49, 86, 62]
[67, 53, 16, 95, 62, 43, 75, 76, 42, 70, 15, 26, 81, 62]
[28, 92, 10, 43, 94, 20, 87, 93, 93, 51, 82, 40, 10, 12]
[75, 67, 28, 37, 11, 46, 21, 82, 83, 55, 90, 97, 13, 54]
[42, 17, 70, 15, 75, 43, 83, 27, 25, 13, 65, 25, 94, 75]
[10, 76, 49, 96, 80, 93, 59, 21, 50, 68, 71, 34, 73, 22]
[16, 25, 19, 96, 12, 45, 48, 12, 77, 39, 79, 39, 38, 16]
----------
25
15
99
29
42
93
83
25
50
77
```